Reflection 1 CSC360

Computer Graphics vs. Image Processing

- **Computer Graphics:** Focuses on generating and rendering new visual content from scratch using basic geometric shapes (points, lines, curves, polygons, and circles).
- **Image Processing:** Involves analyzing, enhancing, and extracting data from pre-existing images.

Version Control Security (Git, SSH, & HTTPS)

- **HTTPS:** Secures web traffic and repository access via TLS encryption.
- **SSH:** Provides secure remote authentication using asymmetric key pairs:
  - **Public Key:** Shared openly to grant access.
  - **Private Key:** Kept confidential on your local machine.

Java Graphical Frameworks

- **AWT:** Uses native OS elements for rendering interfaces.
- **Swing:** Built on top of AWT to offer more flexible, lightweight components.
- **JavaFX:** Modern framework designed for rich interfaces, animations, custom styling, and multimedia integration.

Interaction & Event Handling

- **Event Listener Model:** Handles user input through a connected workflow: an **event source** triggers an **event** (e.g., a button click), which is then processed by a registered **listener**.
- **Interactive Graphics:** Dynamic visuals that react to mouse movements, clicks, or keyboard events by triggering event handlers and repainting the display.

Digital Images & Graphics Types

- **Digital Images:** Grid matrices where individual pixels hold specific intensity (grayscale) or color (RGB) values.
- **Raster Graphics:** Grid-based pixel displays where image quality depends on resolution; scaling them up causes pixelation and loss of clarity.
- **Vector Graphics:** Scalable shapes defined by mathematical formulas rather than fixed pixels, making them resolution-independent.
- **Display Mechanics:** Monitors plot basic line primitives by choosing the optimal arrangement of rows and columns of pixels.

Mathematics in Graphics

- **Calculus & Curves:**
  - **Differentiation:** Calculates slopes and rates of change to plot smooth curves.
  - **Integration:** Computes areas beneath or between curved lines.
- **2D Transformations:**
  - **Translation:** Shifting an object's position on the screen.

    - **Scaling:** Resizing objects to simulate zooming in or out.

Practical Java 2D Applications

- Render basic shapes (e.g., a triangle) on a screen.
- Bind mouse movement events to the shape's coordinates to allow dynamic repositioning.

Operating Systems & Display Differences

- **Windows vs. Linux:** Visual outputs vary across operating systems due to differences in font rendering techniques and graphics driver compatibility.

Summary of Learning Outcomes

This class covered foundational graphics concepts—including geometric primitives, image structures, raster vs. vector formats, mathematical modeling, and Java rendering frameworks. The hands-on exercise of building an interactive, mouse-controlled triangle demonstrated how event handling and 2D transformations translate theoretical concepts into functional graphics programming.

Action Items / Revision Checklist

- [ ] Contrast SSH vs. HTTPS protocols.
- [ ] Review SSH public/private key dynamics.
- [ ] Compare AWT, Swing, and JavaFX features.
- [ ] Practice coding the Java Event Listener Model.
- [ ] Contrast raster and vector graphics behavior.
- [ ] Practice applying translation and scaling math.
- [ ] Build a Java 2D program rendering a triangle.
- [ ] Implement mouse-driven motion for graphics objects.
- [ ] Review pixel-level line rasterization.
- [ ] Revisit calculus applications for smooth curves.
- [ ] Compare Windows and Linux display architectures.